

OLGA CONNECT

NLP Service Architecture

Standalone API for intent processing, reciprocal matching and explainable recommendations

Document	Status	Scope
Architecture document	Draft v1.0	NLP service only; application and database details intentionally minimized

RECOMMENDATION Build NLP as a separate backend API. The caller sends structured offer/need input; the service returns a ranked, explainable result. The mobile app never calls the embedding provider directly.

1. Purpose and boundaries

The NLP service converts professional offers and needs into normalized text, embeddings, reciprocal relevance scores, reason codes and a safe JSON response. It is independently testable before the mobile application is complete.

In scope

- Validation, normalization, language and obvious PII checks
- Embedding generation for offer and need text
- Reciprocal semantic comparison in both directions
- Hybrid ranking using semantic and structured signals
- Deterministic explanation templates
- Feedback capture and offline evaluation

Out of scope

- Authentication, consent, blocking and final visibility decisions
- Profiles, chat, notifications, location tracking and transactions
- Chatbots, autonomous agents or LLM-generated explanations
- Custom model training or fine-tuning in V1

2. Logical architecture

OLGA Connect NLP Service - Component Architecture

Separate API boundary with an internal candidate repository in V1

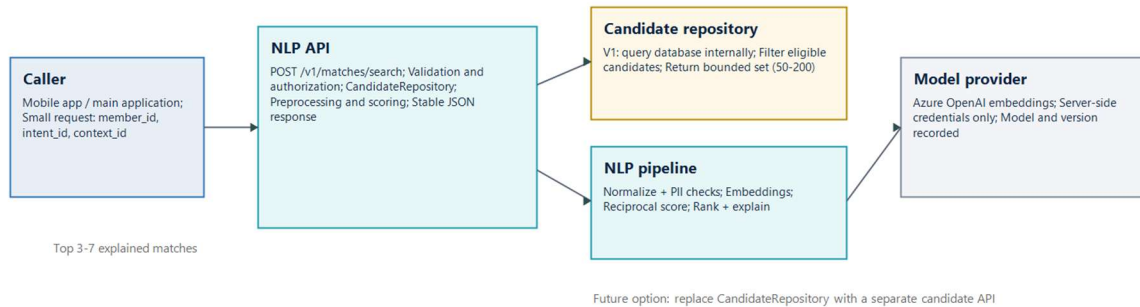


Figure 1A. Component and deployment boundary for the standalone NLP service.

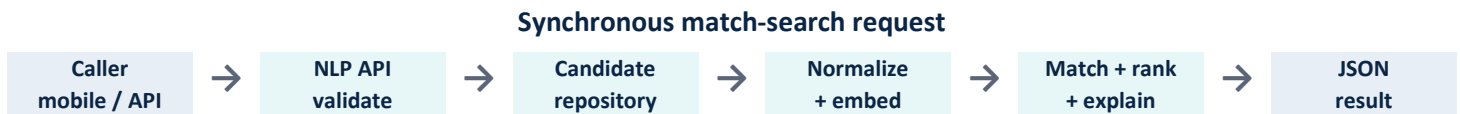


Figure 1. Request path from caller to ranked, explainable JSON response.

Component	Responsibility	Output
NLP API	Validate request, enforce service auth and return stable response	HTTP response
Candidate repository	Initially queries the database for a small, eligible candidate set; later may become another API	Filtered candidates
Preprocessor	Normalize text, detect language and check PII/safety	Normalized intent
Embedding adapter	Create or reuse model vectors	Versioned embedding
Matcher/ranker	Compare reciprocal directions and apply weights	Ranked score
Explanation engine	Map signals to approved templates	Reason codes/text
Evaluation module	Compare results with labels and feedback	Metrics/error report

3. API contract

Use a versioned REST contract. The production operation is a one-user match search: the API receives the requesting member and context, retrieves a filtered candidate set internally, then scores and ranks that set. Keep the embedding provider behind an adapter so local experiments and Azure-hosted execution use the same API.

Field	Type	Required	Description
request_id	string	Yes	Trace and safely retry a request

member_id	string	Yes	Requesting member identifier
intent_id	string	Yes	Current offer/need intent identifier
context_id	string	Yes	Matching context identifier
limit	integer	No	Maximum final results, default 7
options	object	No	Threshold and language options

Example request

```
{ "request_id": "req-1001", "member_id": "A123", "intent_id": "intent-789", "context_id": "event-001", "limit": 7 }
```

Example response

```
{ "request_id": "req-1001", "matches": [{ "member_id": "B456", "score": 0.87, "label": "strong_match", "reason_codes": ["CATEGORY_COMPLEMENT", "INDUSTRY_MATCH"], "reason_text": "This member offers cold-chain warehousing relevant to your need." }], "model_version": "embedding-v1", "ranking_version": "ranking-v1" }
```

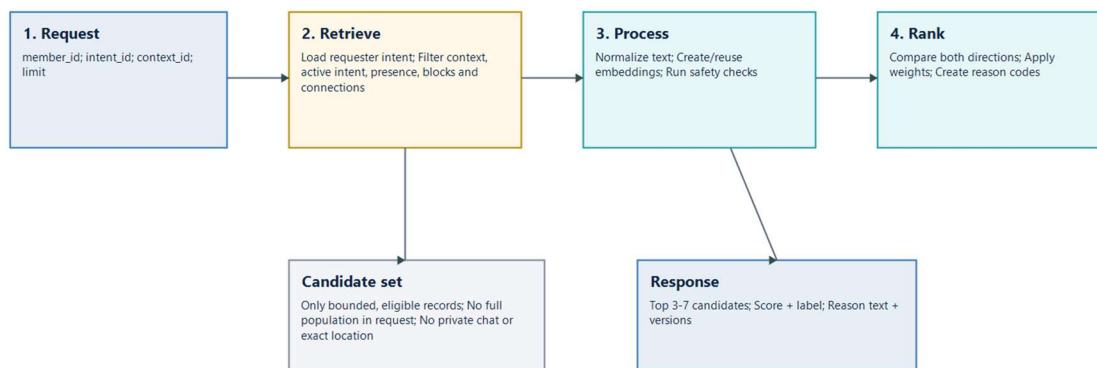
BOUNDARY A high NLP score is only a recommendation. The caller remains responsible for consent, blocking, visibility, eligibility and user-facing actions.

4. Processing flows

4.1 Synchronous matching

OLGA Connect NLP Service - Request and Data Flow

One requesting member in the API request; candidates are retrieved internally



Application applies final consent and visibility rules before display

Figure 2A. Request and data flow with internal candidate retrieval.

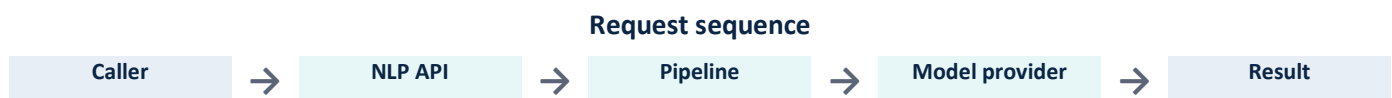


Figure 2. Synchronous request sequence.

1. Caller sends POST /v1/matches/search with member_id, intent_id and context_id.
2. API validates schema, service authorization, limits and request_id.
3. Candidate repository queries the database for a small eligible candidate set.
4. Preprocessor normalizes the requester and candidate intent text.
5. Embedding adapter creates or retrieves offer/need embeddings.
6. Matcher calculates reciprocal need-to-offer scores for each candidate.
7. Ranker applies configured weights and structured signals.
8. Explanation engine returns reason codes and approved text for the top results.
9. API returns only the final 3–7 matches plus model and ranking versions.

4.2 Candidate retrieval boundary

For V1, the NLP API calls an internal CandidateRepository that queries the database. The repository must filter by active context, active and unexpired intent, live presence, visibility, block status and existing connection status before NLP scoring. It should return a bounded set, such as the top 50–200 candidates, rather than the entire user population. Later, CandidateRepository can be implemented as a separate API without changing the external NLP contract.

4.3 Embedding refresh

When intent text changes, embedding generation can run asynchronously. Retain the last approved version until the new version is validated. This keeps matching available during provider failures or model changes.

5. NLP design

Normalization

- Preserve original text for display and audit.
- Normalize whitespace, Unicode variants and punctuation.
- Detect language and return a clear unsupported-language error.
- Mask obvious emails, phone numbers and unnecessary personal data.
- Keep negation, quantities and locations; do not over-clean the text.
- Create a normalized-text hash for caching and reproducibility.

Embedding and reciprocal scoring

Create one embedding for each offer and need. Compare both directions and combine them using a harmonic mean when mutual benefit is required.

Signal	Starting weight	Purpose
Category complementarity	0.40	Enforce offer/need relationship
Reciprocal semantic similarity	0.25	Capture paraphrases and related meaning
Industry/function adjacency	0.15	Reduce broad irrelevant matches

Seniority compatibility	0.10	Reduce implausible pairings
Freshness	0.10	Prefer current intent

Explanation

Generate explanations only from signals that contributed to the score. Use approved deterministic templates in V1. If no valid explanation can be produced, suppress the match.

6. Security and reliability

Control	Required behavior
Credentials	Embedding keys stay server-side; never ship them in the mobile app.
Input safety	Validate size/type, rate-limit requests and reject abusive payloads.
Data minimization	Send only normalized intent and necessary context to the model provider.
Versioning	Return preprocessing, model and ranking versions on every result.
Retries	Use request_id, timeouts, bounded retries and degraded responses.
Observability	Log request_id, latency, versions and outcome; avoid raw text by default.
Rollback	Switch active model or ranking version after evaluation failure.

7. NLP data model and database procedures

The NLP service needs only a focused set of tables. The wider OLGA application may own member, context and consent tables; the CandidateRepository reads the minimum fields needed to filter and score candidates.

Table	Purpose	Key fields
NlpIntent	Current offer/need text submitted for matching	intent_id, member_id, context_id, intent_type, original_text, normalized_text, status, expires_at
NlpEmbedding	Versioned vector for an intent	intent_id, model_version, dimensions, normalized_hash, embedding, created_at
NlpRankingConfig	Active scoring weights and thresholds	ranking_version, weights_json, threshold, active_from, active_to
NlpMatchResult	Returned or cached ranked result	request_id, requester_id, candidate_id, score, reason_codes, reason_text, ranking_version
NlpFeedback	Human feedback for evaluation	request_id, requester_id, candidate_id, label, reason, created_at
NlpModelVersion	Provider and preprocessing metadata	model_version, provider, dimensions, preprocessing_version, active

Logical database view

NLP data relationship



NlpIntent 1-to-many NlpEmbedding; one request produces many NlpMatchResult rows; feedback references the request and candidate result. Keep original text separate from normalized text, and delete or anonymize records according to the approved retention policy.

V1 database access boundary



Representative stored procedures

The procedures below are intentionally narrow. They keep candidate filtering and result persistence transactional while leaving the semantic scoring implementation in the NLP service.

```

CREATE PROCEDURE nlp.GetRequesterIntent
    @MemberId varchar(64), @IntentId varchar(64), @ContextId varchar(64)
AS
SELECT TOP (1) * FROM nlp.NlpIntent
WHERE intent_id=@IntentId AND member_id=@MemberId AND context_id=@ContextId
    AND status='ACTIVE' AND expires_at > SYSUTCDATETIME();

CREATE PROCEDURE nlp.GetEligibleCandidates
    @RequesterId varchar(64), @ContextId varchar(64), @MaxRows int = 200
AS
SELECT TOP (@MaxRows) i.intent_id, i.member_id, i.intent_type,
    i.normalized_text, i.expires_at
FROM nlp.NlpIntent i
WHERE i.context_id=@ContextId AND i.status='ACTIVE'
    AND i.expires_at > SYSUTCDATETIME()
    AND i.member_id <> @RequesterId
ORDER BY i.expires_at DESC;

CREATE PROCEDURE nlp.SaveMatchResults
    @RequestId varchar(64), @RequesterId varchar(64), @ResultsJson nvarchar(max)
AS
-- Validate request_id uniqueness, insert ranked results and return saved rows.
-- The API supplies reason codes, score and ranking/model versions.
  
```

IMPLEMENTATION NOTE Use stored procedures for candidate retrieval and durable result persistence. Keep embeddings, cosine calculations and ranking rules behind application interfaces so the scoring engine remains testable outside SQL.

8. .NET data-access approach

Use a deliberate hybrid rather than choosing EF Core or raw ADO.NET for every operation. EF Core remains the default data-access tool; Microsoft.Data.SqlClient is the escape hatch for SQL Server-specific operations.

Concern	Use	Why
Schema and migrations	EF Core migrations	Versioned schema changes and repeatable environments
Normal CRUD	EF Core DbContext	Less boilerplate and clear unit-of-work behavior

Ordinary relational reads	EF Core LINQ with AsNoTracking	Readable, parameterized queries
Candidate retrieval	Stored procedure called through EF Core or SqlClient	Centralized filtering and bounded result set
Vector distance / vector columns	Microsoft.Data.SqlClient	Direct access to SQL Server-specific vector behavior
Transactions and result persistence	EF Core transaction or stored procedure	Atomic writes and retry-safe persistence
Performance tuning	Stored procedure or parameterized SQL	Use only when measured query plans justify it

Where each approach is used

- Use EF Core for NlpIntent, NlpMatchResult, NlpFeedback, NlpModelVersion and NlpRankingConfig entities, migrations, ordinary CRUD and transaction coordination.
- Use CandidateRepository as the only component allowed to retrieve candidates. It calls nlp.GetRequesterIntent and nlp.GetEligibleCandidates, so filtering is centralized and testable.
- Use Microsoft.Data.SqlClient directly when reading/writing Azure SQL vector values or when EF Core cannot express a vector-specific query cleanly.
- Do not put SQL calls in controllers, mobile code or the NLP scoring classes. Controllers call MatchingService; MatchingService calls CandidateRepository.
- Do not add Dapper in V1. Add it only if profiling shows a clear benefit and the team accepts another data-access abstraction.

Recommended project boundary

Api/Controllers → Application/MatchingService → Infrastructure/CandidateRepository → EF Core / Stored Procedures / Microsoft.Data.SqlClient → Azure SQL

Connection and security

Use Microsoft Entra authentication and managed identity from the hosted service to Azure SQL. Local development can use DefaultAzureCredential. Keep connection creation in one infrastructure registration and use connection pooling; do not create a new database strategy per request or store SQL passwords in source control.

Representative .NET registration

```
builder.Services.AddDbContext<NlpDbContext>(options =>
    options.UseSqlServer(configuration.GetConnectionString("AzureSql")));

builder.Services.AddScoped<ICandidateRepository, CandidateRepository>();
builder.Services.AddScoped<IMatchingService, MatchingService>();
```

Representative repository call

```
public async Task<IReadOnlyList<CandidateRow>> GetEligibleAsync(
    string requesterId, string contextId, int maxRows, CancellationToken ct)
{
    return await db.CandidateRows
        .FromSqlInterpolated($"EXEC nlp.GetEligibleCandidates" +
```

```

        $" @RequesterId={requesterId}, @ContextId={contextId}, @MaxRows={maxRows}"
    ).AsNoTracking()
    .ToListAsync(ct);
}

```

SENIOR ARCHITECT DECISION EF Core is the default. Stored procedures and Microsoft.Data.SqlClient are intentional, localized exceptions for candidate retrieval, vector operations and high-value SQL-specific behavior. This keeps the code maintainable without hiding important database performance characteristics.

9. Testing and evaluation

- Unit tests for normalization, PII masking, reciprocal scoring and reasons.
- Contract tests for request validation, response shape, errors and version fields.
- Golden tests with known intent pairs and expected reason codes.
- Fake-provider tests so CI does not depend on Azure availability.
- Load tests for latency, concurrency, timeout and retry behavior.
- Privacy tests confirming prohibited fields never reach the provider.

Create at least 200 anonymized labelled pairs: strong, plausible, weak, no match and unsafe/ineligible. Split by member or organization. Track Precision@5, Recall@20, reciprocal precision, request rate, acceptance rate, not-useful rate and explanation coverage.

10. Recommended implementation phases

Phase	Deliverable	Exit criterion
1. Local baseline	Python module, JSON/CSV input, normalizer and local embeddings	Repeatable notebook and tests
2. Matching core	Reciprocal score, configurable weights and reason catalog	Golden rankings and explanations pass
3. API wrapper	POST /v1/match, validation, errors, versions and health endpoint	Contract tests pass
4. Hosted provider	Azure embedding adapter with versioned configuration	Provider swap does not change contract
5. Evaluation loop	Labels, metrics, error review and model card	Baseline and rollback approved
6. Pilot tuning	Real feedback and threshold/weight tuning	Quality improves without safety loss

11. Final decision

DECISION NLP is a separately deployable backend service with a stable REST contract. The main production operation accepts one requesting member, intent and context; the service initially queries an internal CandidateRepository, scores a bounded candidate set, and returns the top explained matches. Pairwise scoring remains available only for testing and evaluation. The repository can later move behind another API without changing the NLP contract.